

Complete Beginner Guide: Dockerize and Host a Python App on Render

This guide explains step-by-step how to:

- Understand Docker basics
- Containerize a Python Flask application
- Build Docker images
- Run containers locally
- Push code to GitHub
- Deploy the application on Render

1. What is Docker?

Docker is a platform used to package applications into containers.

A container includes:

- Your application code
- Python runtime
- Libraries and dependencies
- Configuration

This ensures your application works consistently on every system.

2. What is an Image?

A Docker image is a blueprint or template.

Examples:

- python:3.11
- ubuntu
- nginx

Images are downloaded from Docker Hub and used to create containers.

3. What is a Container?

A container is a running instance of an image.

Think of it this way:

- Image = Recipe
- Container = Prepared food

Containers isolate applications from the host operating system.

4. Install Docker Desktop

Download Docker Desktop from:

<https://www.docker.com/products/docker-desktop/>

After installation:

1. Start Docker Desktop
2. Open terminal
3. Verify installation:

```
docker --version
```

5. Your Python Project Structure

Example Flask project structure:

```
project/
├──
├── app.py
├── requirements.txt
├── Dockerfile
└── .dockerignore
```

6. Create requirements.txt

This file contains all Python dependencies.

Command:
pip freeze > requirements.txt

Example contents:
flask==3.0.0
gunicorn==22.0.0

7. Create app.py

Example Flask application:

```
from flask import Flask
import os

app = Flask(__name__)

@app.route("/")
def home():
    return "Website Running Successfully"

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    app.run(host="0.0.0.0", port=port)
```

Explanation:

- host="0.0.0.0" allows external access
- PORT environment variable is required for Render deployment

8. Create Dockerfile

Dockerfile tells Docker how to build the image.

Example:

```
FROM python:3.11

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .
EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

9. Dockerfile Explanation

```
FROM python:3.11
Downloads official Python image.

WORKDIR /app
Creates working directory inside container.

COPY requirements.txt .
Copies dependency file into container.

RUN pip install
Installs all Python packages.

COPY . .
Copies project files into container.

EXPOSE 5000
Container listens on port 5000.

CMD ["python", "app.py"]
Runs Flask application when container starts.
```

10. Create .dockerignore

Example:

```
venv
__pycache__
.git
.env
```

Purpose:

Prevents unnecessary files from entering the container.

11. Build Docker Image

Command:

```
docker build -t mypythonapp .
```

Explanation:

- `docker build` = start image build
- `-t mypythonapp` = image name
- `.` = current directory contains Dockerfile

Internally Docker:

1. Downloads Python image
2. Installs dependencies
3. Copies project files
4. Saves everything as image

12. Verify Images

Command:

```
docker images
```

Shows all Docker images available on your system.

13. Run Docker Container

Command:

```
docker run -p 5000:5000 mypythonapp
```

Explanation:

- `docker run` = create container
- `-p 5000:5000` = port mapping
- `mypythonapp` = image name

Meaning:

Laptop port 5000 connects to container port 5000.

14. Open Website

Open browser:

```
http://localhost:5000
```

If website opens successfully, your container works.

15. Push Project to GitHub

Commands:

```
git init
git add .
git commit -m "docker setup"
```

```
git branch -M main
```

```
git remote add origin YOUR_REPO_URL
```

```
git push -u origin main
```

This uploads your project to GitHub.

16. Deploy on Render

Steps:

1. Open Render dashboard
2. Click New → Web Service
3. Connect GitHub repository
4. Select Environment = Docker
5. Deploy application

Render automatically:

- Pulls GitHub repository
 - Reads Dockerfile
 - Builds image
 - Runs container
 - Provides public URL
-

17. Common Errors

Application failed to respond:
Usually caused by missing:
host="0.0.0.0"

Build failed:
Usually missing requirements.txt or invalid Dockerfile.

Module not found:
Run:
pip freeze > requirements.txt

18. Full Deployment Flow

```
graph TD
    A[Python App] --> B[Dockerfile]
    B --> C[Docker Image]
    C --> D[GitHub]
    D --> E[Render]
    E --> F[Live Website]
```

This is the standard modern deployment workflow.

Congratulations! You now understand the complete beginner-to-deployment workflow using Docker and Render for Python applications.